

Guide to Breakout Environment Options

The primary manual specifies baseline options to ensure that all students start from the same baseline. This reference document explains the environmental parameters of **ALE/Breakout-v5** and the effects of commonly used wrappers. You may use this information to design your own experiments or to gain a deeper understanding of the baseline's behavior.

The baseline for this project is established with the following configurations to ensure evaluation fairness

```
env = gym.make(  
    "ALE/Breakout-v5",  
    frameskip=1,  
    repeat_action_probability=0.0,  
    full_action_space=False,  
)
```

Comparison with Legacy Environments

This configuration is equivalent to the BreakoutNoFrameskip-v4 environment from OpenAI Gym, which was the predecessor to Gymnasium. Most research papers and academic studies have conducted their experiments using this specific setup. Consequently, a vast majority of GitHub repositories and existing codebases are authored based on these parameters.

Table of Contents (Ctrl+Click to follow link)

- A. [gym.make Arguments \(Environment-Specific Options\)](#)
- B. [Observation Shapes for Training Code](#)
- C. [Common Errors and Troubleshooting](#)

A. gym.make Arguments (Environment-Specific Options)

These arguments are specified when calling `gym.make` and determine the fundamental behavior of the environment. If you wish to use values different from the project baseline, you must explicitly configure them in your code.

A.1 frameskip

- **Default Value:** 4 (Skips 4 frames internally within the environment)
- **Recommended Value:** 1 (No internal frame skipping)

Description: This parameter determines how many frames a single action is repeated for.

- If **frameskip=4**, a single action output by the agent is applied to the environment for 4 consecutive frames.
- If **frameskip=1**, an action is applied at every single frame.

Caution: Sprite Flickering and Max Pooling Due to hardware limitations, the Atari 2600 often renders only a subset of objects in one frame and the remaining objects in the next (known as **sprite flickering**). While the human eye perceives these as continuous due to persistence of vision, an AI agent might miss objects that are not rendered in a single frame.

To address this, a common practice since the original DQN paper (Mnih et al., 2015) is to combine two adjacent frames by selecting the maximum pixel values (**max pooling**) and repeating a single action for 4 frames. Since this processing is typically handled within the code (via wrappers) rather than by the environment itself, the standard approach is to:

1. Set **frameskip=1** to disable the environment's internal skipping.
2. Manually handle the 4-frame repetition and frame merging in the training script.

Warning: If you use external code that assumes `frameskip=1` but leave the environment at its default **frameskip=4**, the agent will effectively only make a decision every 16 frames, which will significantly degrade performance.

Option Formats

- **Integer k:** Skips exactly k frames every time (**Deterministic**).
- **Tuple (a, b):** Skips a random number of frames between a and b (**Stochastic**).

Trade-offs

- **High frameskip:** Faster training (more environment time passes per step), but lower control resolution.
- **Low frameskip:** Enables more precise control, but may reduce training speed and sample

efficiency.

- **Random frameskip:** Can lead to a more robust policy that is less dependent on deterministic patterns.

A.2 repeat_action_probability (Sticky Actions)

- **Default Value:** 0.25
- **Recommended Value:** 0.0 (Disable sticky actions)

Description: This parameter defines the probability P that the agent's chosen action will be ignored in favor of repeating the action from the previous step. The **v5** environment introduced this default probability based on research by Machado et al. (2018), which argued that the deterministic behavior of **v4** allowed agents to simply memorize trajectories rather than learning a generalized policy. However, since the majority of existing research and baselines still utilize the **v4** standard, the recommended value for this project is set to **0**.

Trade-offs

- **0.0 (Recommended):**
 - **Pros:** Training is more stable and converges faster.
 - **Cons:** The agent may exploit deterministic patterns and simply "memorize" the sequence of moves.
 - **Note:** This is the most common setting used in prior research and open-source implementations.
- **0.25 (v5 Default):**
 - **Pros:** Encourages a more robust policy and is the current standard for modern RL benchmark comparisons.
 - **Cons:** Increases training difficulty and environment stochasticity.

A.3 full_action_space

- **Default Value:** False (4 actions for Breakout)
- **Recommended Value:** False

Description:

- **False:** Provides only the set of actions that are meaningful for the specific game. For Breakout, this includes 4 actions: **NOOP**, **FIRE**, **RIGHT**, and **LEFT**.
- **True:** Uses the full standard Atari action space of 18 actions (NOOP, FIRE, 8 directions, and 8 directions + FIRE).

Trade-offs

- **False (Recommended):**
 - **Pros:** A smaller action space leads to faster exploration.
 - **Cons:** Specifically tailored to a single game.
 - **Note:** Ideal for focused training on the Breakout environment.
- **True:**
 - **Pros:** Allows for training multiple Atari games using the same network architecture (multi-game RL).
 - **Cons:** Increases the action space significantly (from 4 to 18), making exploration much more difficult.

A.4 difficulty

- **Default Value:** 0 (Easiest)
- **Description:** Determines the level of difficulty within the same game mode. In Breakout, there are two levels: 0 and 1. Setting this to 1 increases the speed of the ball.

A.5 obs_type

- **Default Value:** "rgb" (210 × 160 × 3 color image)

Options:

- **"rgb":** 210 × 160 × 3 RGB image
- **"grayscale":** 210 × 160 grayscale image
- **"ram":** 128-byte direct RAM state

Trade-offs:

- **rgb / grayscale:**
 - **Pros:** Visual input processed via CNN; a representation close to how a human plays the game.
 - **Note:** It is common to post-process RGB into grayscale to utilize a smaller observation space when using CNNs.
- **ram:**
 - **Pros:** Directly receives the internal state of the game; the input size is very small (128), leading to faster training.

- **Cons:** Fundamentally changes the nature of the task and makes generalization to other games difficult.

A.6 render_mode

- **Default Value:** None

Options:

- **None:** No visual output (Recommended for faster training).
- **"rgb_array":** Returns the screen as a NumPy array (Useful for saving videos or logging).
- **"human":** Visualizes the game in a separate window (Used for debugging or demos; very slow).

Note: This setting does not affect the learning process itself; it only impacts the training speed.

B. Observation Shapes for Training Code

Observations returned by the Gymnasium environment are typically not fed directly into a learning model as-is. Instead, they must be modified to match the input requirements of your specific model. Whether you are writing the code from scratch or using a specialized library, it is crucial to understand the shape of the observations and the processing methods involved to ensure your code functions as intended.

B.1 Raw Values from the Base Environment

These are the values returned by the Gymnasium environment during the step and reset functions.

- **Observation (obs):** Shape **(210, 160, 3)**, dtype **uint8**.
 - This is the raw RGB image at the original Atari resolution, where each pixel has R/G/B values ranging from 0 to 255.
- **Actions:** 4 discrete actions.
 - Integers **0–3 (0: NOOP, 1: FIRE, 2: RIGHT, 3: LEFT)**.
- **Step Return Values (5 items):** obs, reward, terminated, truncated, info.
- **Reset Return Values (2 items):** obs, info.

Detailed Definitions:

- **obs:** The observation mentioned above.
- **reward:** The reward value received in the current step.
- **terminated:** A boolean indicating if the game has ended because all lives were lost.
- **truncated:** A boolean indicating if the game ended due to external factors like time limits.
 - *Note:* The distinction between terminated (MDP internal termination) and truncated (MDP external termination) is significant for learning value functions in reinforcement learning.
- **info:** A dictionary containing auxiliary information for reference.

Why Preprocessing is Needed: Feeding the raw **(210, 160, 3)** image directly into a CNN is computationally inefficient and does not address issues like sprite flickering. Therefore, the following processing steps are typically required.

B.2 Standard Input Values Used in DQN

Since the original DQN paper, virtually all Atari reinforcement learning implementations have adopted the following standard processing:

- **Single Frame:** A 84×84 **grayscale** image (converted from 210×160 RGB via resizing and grayscaling).
- **Frame Stacking:** The most recent **4 frames** are stacked along the channel dimension.
- **Final Observation Shape:** **(4, 84, 84)**, with a dtype of **uint8** (or **float32** if normalized to a 0–1 range).
- **PyTorch Input Shape:** **(batch, 4, 84, 84)**.

Note on Preprocessing: Standard practice also involves applying the previously mentioned **max pooling** (combining two adjacent frames) before these transformations.

If you are using libraries that support Atari environments, such as **Stable Baselines3 (SB3)** or **CleanRL**, these preprocessing steps are usually handled automatically. In most cases, you will not need to manually perform these dimensional transformations yourself.

B.3 Migrating from Legacy Gym / BreakoutNoFrameskip-v4 Code

When utilizing older codebases from sources like GitHub, you will frequently encounter these critical differences. Updating your environment is not as simple as a name replacement; switching from BreakoutNoFrameskip-v4 to ALE/Breakout-v5 without proper configuration changes the structure of the data received, effectively changing the nature of the task itself.

```
# Legacy Gym
env = gym.make("BreakoutNoFrameskip-v4")
# Equivalent call in Gymnasium
env = gym.make("ALE/Breakout-v5",
               frameskip=1,
               repeat_action_probability=0.0,
               full_action_space=False)
```

API Changes: reset and step

Legacy Gym (Old):

```
obs = env.reset() # Returns 1 value
obs, reward, done, info = env.step(action) # Returns 4 values
```

Gymnasium (Current):

```
obs, info = env.reset() # Returns 2 values
obs, reward, terminated, truncated, info = env.step(action) # Returns 5 values
# Convert terminated and truncated into a single 'done' flag
done = terminated or truncated
```

Failure to update these two parts often leads to ambiguous errors, such as a **ValueError** (due to unpacking the wrong number of values) or **unexpected freezes** during training.

B.4 Using Stable-Baselines3 (SB3)

Stable-Baselines3 provides a convenient helper function that applies standard Atari preprocessing in a single line. You can create the baseline environment using the following example code:

```
from stable_baselines3.common.env_util import make_atari_env
from stable_baselines3.common.vec_env import VecFrameStack
env = make_atari_env(
    "ALE/Breakout-v5",
    n_envs=8,
    env_kwargs={
        "frameskip": 1,
        "repeat_action_probability": 0.0,
        "full_action_space": False,
    },
)
env = VecFrameStack(env, n_stack=4)
```

You can specify settings within the `make_atari_env` function. If these parameters are omitted, SB3 will revert to the **v5 default settings** (`frameskip=4`, sticky action probability of 0.25).

The **`VecFrameStack(env, 4)`** function handles the stacking of the last 4 frames for you. Since SB3's **`CnnPolicy`** is designed to automatically process **(4, 84, 84)** inputs, students do not need to manually reshape or transform tensor dimensions.

B.5 When Using CleanRL Code

In CleanRL's single-file implementations, such as `ppo_atari.py`, there is typically a dedicated `env_id` variable, making it easy to change the environment name. However, simply updating the name is not enough.

To match the project baseline, you must ensure that the `gym.make(env_id)` call within the script includes the baseline arguments: **`frameskip=1`**, **`repeat_action_probability=0.0`**, and **`full_action_space=False`**.

CNN Architecture and Preprocessing: The CNN definitions in CleanRL are designed with the assumption of **4 input channels** (for the 4-frame stack) and an input size of **84×84** .

- *Example:* `nn.Conv2d(4, 32, 8, stride=4)`

The safest approach is to leave the internal logic for 4-frame repetition, frame max-pooling, grayscale conversion, and 84×84 resizing intact, and **only modify the environment initialization part** of the code.

B.6 Verifying Observation Shapes and Dimensional Permutation

While most Atari-supporting packages handle dimensional transformations automatically, you must be extremely careful if you are writing custom code or using raw implementations. One of the most common errors in reinforcement learning occurs when the order of dimensions in the observation does not match what the model expects. Specifically, when feeding image inputs into a CNN, you must precisely align the order of the **height**, **width**, **channel**, and **batch** dimensions.

To avoid these issues, it is highly recommended to always print and verify the shapes as follows:

```
print(type(obs))
print(obs.shape)
print(obs.dtype)
```

The most common observation shapes you will encounter are as follows:

- **Base ALE/Breakout-v5:**
 - `obs.shape = (210, 160, 3)`
 - **Meaning:** height=210, width=160, RGB channels=3
- **Standard DQN Atari Input:**
 - `obs.shape = (4, 84, 84)`
 - **Meaning:** A stack of the 4 most recent 84×84 grayscale frames.
- **PyTorch CNN Input:**
 - `x.shape = (batch, 4, 84, 84)`
 - **Meaning:** [batch size, number of channels/frames, height, width]

Important Note on Batch Dimensions: If a single observation from your environment has a shape of **(4, 84, 84)**, you must add a batch dimension before feeding it into a PyTorch model, resulting in a shape of **(1, 4, 84, 84)**.

```
obs, info = env.reset()

# If obs is (4, 84, 84)
x = torch.as_tensor(np.array(obs), dtype=torch.float32)
x = x.unsqueeze(0)      # (4, 84, 84) -> (1, 4, 84, 84)
x = x / 255.0           # uint8 0~255 -> float32 0~1
```

Some libraries and frameworks handle image dimensions in the order of **(height, width, channel)**. This is known as the **Channel-Last** format.

- *Example:* (84, 84, 4)

Conversely, PyTorch's Conv2d layers expect the dimensions in the order of **(channel, height, width)**. This is known as the **Channel-First** format.

- *Example:* (4, 84, 84)

How to Reorder Dimensions

```
# channel-last: (84, 84, 4)
# channel-first: (4, 84, 84)

obs = np.transpose(obs, (2, 0, 1))
```

Including the batch dimension

```
# obs: (84, 84, 4)
x = torch.as_tensor(obs, dtype=torch.float32)

x = x.permute(2, 0, 1) # (84, 84, 4) -> (4, 84, 84)
x = x.unsqueeze(0)     # (4, 84, 84) -> (1, 4, 84, 84)
x = x / 255.0          # Converting from integer to float
```

C. Common Errors and Troubleshooting

1. Expected 4D input to conv2d

- **Issue:** This occurs when the batch dimension is missing. PyTorch CNNs require a 4D input of **(N, C, H, W)**, but you likely provided a 3D input **(C, H, W)**.
- **Solution:** Use **x.unsqueeze(0)** to add the batch dimension.

2. Expected input to have 4 channels, but got 84 channels

- **Issue:** This is a classic dimensional ordering error. You likely fed a **Channel-Last (B, H, W, C)** tensor into a PyTorch CNN, which interpreted the height (**84**) as the number of channels.
- **Solution:** Move the channel dimension to the front using **permute** or **transpose**.

3. Training runs, but performance does not improve

- **Issue:** You might be using only a single frame, or **Frame Stacking** might not be applied correctly. Without stacked frames, the agent cannot perceive the direction or speed of the ball.
- **Solution:** Check your **obs.shape**. Verify if it is **(4, 84, 84)** (correct) or just **(84, 84)** (missing temporal context).

4. Input values are too large

- **Issue:** Raw uint8 images range from **0 to 255**. Using these values directly can cause unstable gradients or saturated activation functions.
- **Solution:** When implementing custom training loops, always convert the data to **float32** and divide by **255.0** to normalize the input range to **[0.0, 1.0]**.